daffi

version 1.4.1

None

None

Table of contents

1. Overview 3
2. Installtion 4
   2.1 pip 4
3. Usage 5
   3.1 Base objects 5
   3.2 Global 6
   3.3 callback decorator 9
   3.4 fetcher decorator 11
   3.5 Execution modifiers 13
   3.6 Basic example 15
   3.7 Example with bidirectional communication 19
4. Node and Controller 22
   4.1 Controller 22
   4.2 Node 22
5. Asynchronous applications 24
6. Advanced 25
   6.1 G object injection 25
   6.2 Callback classes 26
   6.3 Broadcasting 28
   6.4 Pipeline and g object injection 30
   6.5 Scheduling tasks 32
   6.6 Streams 34
7. Code reference 38
   7.1 Global 38
   7.2 callback 41
   7.3 fetcher 42

1. Overview

Test and validate passing | Publish documentation passing | Coverage 77% | License MIT | code style black | os linux | os macOS | downloads/month 1k

Daffi's goal is to move away from the traditional server-client communication approach where processes are unequal. This concept has been replaced by the controller-node principle. Each process can contain either a controller or a node, or both. Each daffi node gets the ability to create remote callbacks and execute callbacks created on other nodes.

This can best be described as a multiprocessing observer pattern where there are remote subscribers to a callback and there are publishers who can trigger those callbacks remotely. This approach allows you to create both simple architectural solutions such as one-to-one communication between nodes and more complex schemes such as a pipeline chain or one-to-many broadcasting. It is worth to note that the daffi's syntax is very easy. Unlike many other similar libraries, on the surface, daffi contains only a few top-level classes for initializing the process as a node or controller.

Briefly speaking about the area of use. Daffi takes the best of several worlds and can be used as a task dispatcher or for streaming.

1.0.1 Features

- Centralized approach to registering callbacks in microservices. You do not need to run dozens of microservers and register callbacks separately for each, which can turn into a difficult to manage architecture.
- Super fast and strong serialization/deserialization system based on grpc streams and dill. You can serialize dataclasses, functions (with yield statements as well), lambdas, modules and many other types.
- Daffi works equally well with both synchronous and asynchronous applications. You can call asynchronous remote callback from synchronous application and vice versa. Trio support is also included.
- Simple syntax. Calling remote callback is as simple as execution of local method.
- Daffi can work via TCP or via UNIX socket.
- Daffi can either manage single on demand tasks or be using for streaming.

2. Installtion

2.1 pip

Daffi is available on PyPI and can be installed with pip.

```
pip install daffi
```

3. Usage

3.1 Base objects

Note

in this section of the documentation we will cover only the basic description and basic syntax of daffi objects. To learn more about the use of a particular object, see the dedicated sections of this documentation.

If you want to look at practical examples of how to use daffi check example and example with bidirectional communication sections of this documentation.

At the top level daffi consists of Global object, two decorators callback and fetcher and execution modifiers.

Daffi objects definition

Daffi object	Description	Example

Global	Global object is the main daffi configurator.
	This is the place where you can specify what to initialize in particular process (Controller, Node or both), what kind of connection you want to have (Unix socket or TCP), specify host/port or UNIX socket path and so on.
	Global must be initialized at application startup.
		`from daffi import Global`
		
		`g = Global(host="localhost", port=8888)`

callback	callback decorator is used when you want to expose function or class ir order to make it visible for other processes (daffi nodes).
	In other words callback registers decorated function/class as remote callback so that you can trigger it from remote.
		`from daffi import callback`
		
		`@callback`
		`def my_callback(arg1: int, arg2: int):`
		`  return arg1 + arg2`

fetcher	fetcher decorator describes how to execute remote callback. For example if you registered function `abc` in process `A` as callback then in process `B` you need to register fetcher with the same name `abc` to execute `abc` from process `A` like it is local function.
	fetcher works in pair with execution modifiers classes.
		`from daffi import fetcher, __body_unknown__`
		
		`@fetcher`
		`def my_callback(arg1: int, arg2: int):`
		`  __body_unknown__(arg1, arg2)`

execution modifiers	execution modifiers are set of classes that describe how to execute remote callback. Possible options you can consider is trigger callback in foreground (wait for result), background (allow process to wait result in background), use broadcast (trigger several callbacks with the same name registered in different processes at once), use stream to remote callback/callbacks and more.	`from daffi import fetcher, __body_unknown__, BG`
		
		`@fetcher(BG(return_result=True, eta=2))`
		`def my_callback(arg1: int, arg2: int):`
		`  __body_unknown__(arg1, arg2)`

3.2 Global

Global is the main initialization daffi entrypoint.

GLOBAL INITIALIZATION

You can initialize Global object using the following syntax:

```
from daffi import Global
g = Global(process_name='my awersome process', init_controller=True, host='localhost', port=8888)
```

where:

`process_name` is optional `Node` identificator. Other nodes see initialized node by its `process_name`

If `process_name` argument is omitted then randomly generated name will be used. But in some cases it is helpful to give nodes meaningful names. For example one process can wait another process by its name:

```
g.wait_process('node name here')
```

`init_controller=True` Means we want to start `Controller` in this process.

`init_node` argument is True by default so if you want to start only controller in particular process you should be explicit:

```
g = Global(process_name=process_name, init_controller=True, init_node=False, host='localhost', port=8888)
```

`host` and `port` arguments give `Controller` and `Node` information how to connect and communicate.

`host` and `port` arguments are also optional. For instance you can specify only `host`. In this case `Controller` / `Node` or both will be connected to random port

You can also skip these two arguments:

```
g = Global(process_name=process_name, init_controller=True)
```

In this case `Controller` / `Node` will be connected using UNIX socket. By default UNIX socket is created within path

```
< temp directory >/daffi/.sock
```

Where `< temp directory >` is temporary directory of machine where `Controller` / `Node` is running. For instance it is going to be `/tmp/daffi/.sock` on Ubuntu.

You can also provide your own directory for UNIX socket:

```
g = Global(process_name=process_name, init_controller=True, unix_sock_path="/foo/bar/biz")
```

EXECUCUTION WORKFLOW

After initialization Global object starts `Controller` / `Node` or both in separate thread.

You can join this thread to main process using `.join` of `.join_async` methods of Global

```
g = Global(process_name=process_name, init_controller=True, init_node=False, host='localhost', port=8888)
g.join()

#--- or `join_async` if your application is asynchronous
await g.join_async()
```

`Controller` / `Node` or both can be stopped using `.stop` method of Global. This method is suitable for short living jobs for example when you want to start daffi process, trigger couple of callbacks on other nodes and terminate process.

```
g = Global(process_name=process_name, init_controller=True, init_node=False, host='localhost', port=8888)
# .....
# Execute remote callbacks ...
g.stop()
```

For this reason Global can be used as context manager. `.stop` method is executed explicitly on exit from context manager scope.

```
with Global(process_name=process_name, init_controller=True, init_node=False, host='localhost', port=8888) as g:
    # .....
    # Execute remote callbacks ...
```

> ⚠ **Warning**
>
> Dont use `.stop` method to stop daffi components in long living applications eg web servers, background workers etc. start/stop daffi components requires some initialization time and resources.

WAITING FOR NODES OR METHODS TO BE AVAILABLE

Sometimes nodes start at different times and because of this, some remote callbacks may not be available immediately.

Global has several methods to control waiting for callbacks availability.

The 2 examples below illustrate waiting for a remote node to be available:

```
g.wait_process('name of remote node')
```

or

```
await g.wait_process_async('name of remote node')
```

Global can also wait a specific callback to be available by its name:

```
g.wait_function('name of remote callback')
```

or

```
await g.wait_function_async('name of remote callback')
```

Waiting by callback name criteria can be useful when many nodes contain a callback with the same name and we need to wait for the presence of one of them

TRANSFER AND EXECUTE FUNCTION ON REMOTE NODE

This option is suitable when you want to create remote callback dynamically and execute them on remote nodes.

Example:

process 1

```
import logging
from daffi import Global

logging.basicConfig(level=logging.INFO)


def main():
    Global(process_name="other_process", init_controller=True, host="localhost", port=8888).join()


if __name__ == "__main__":
    main()
```

(This script is complete, it should run "as is")

process 2

```
import logging
from daffi import Global

logging.basicConfig(level=logging.INFO)


async def func_to_transfer():
    """Return pid id of remote process"""
    import os
```

```
    return os.getpid()


def main():

    with Global(host="localhost", port=8888) as g:

        remote_pid = g.transfer_and_call(remote_process="other_process", func=func_to_transfer)
        print(f"Remote process pid: {remote_pid}")


if __name__ == "__main__":
    main()
```

(This script is complete, it should run "as is")

On example above we transfer function `func_to_transfer` from process2 to process1 and execute it. The result of function execution will be returned to process2.

> **Warning**
>
> You should make sure all imports that are using in function body are available on remote process. Otherwise you should consider import modules in function body (like `import os` in example above).

WORKING WITH SCHEDULED TASKS

Global has methods `get_scheduled_tasks` and `cancel_scheduled_task_by_uuid` to works with scheduled tasks on remote Node. To know more about scheduled tasks go here

- `get_scheduled_tasks` get all scheduled tasks on remote Node by Node name. This method returns all task uuids that are currently scheduled on remote Node.
- `cancel_scheduled_task_by_uuid` cancel one scheduled task on remote Node by its uuid. All uuids can be obtained using `get_scheduled_tasks` method.

REMOTE CALLBACKS EXECUTION WITH G OBJECT

> **Warning**
>
> This syntax is appropriate for tiny microservices only. In general you should consider using fetcher decorator to trigger remote callback as it is more self explanatory syntax option.

once you have the `g` object initialized you can call the remote callback. The general syntax is:

```
...
g.call.< remote callback name >(*args, **kwargs) & < execution modifier >
```

- `remote callback name` is the name of function registered as remote callback on different `Node`
- `execution modifier` is modifier class name that determines how to execute remote callback and what to do with result. More details about execution modifiers here

3.3 callback decorator

callback decorator registers function or class as remote callback. This way function (or all publicly available methods of decorated class) become visible for all other nodes by its name. callback decorator can register both synchronous and asynchronous functions/methods

```
from daffi import callback

@callback
def sum_two_numbers(a: int, b: int) -> int:
    return a + b


@callback
async def subtract_two_numbers(a: int, b: int) -> int:
    return a - b
```

in example above we registered callbacks with names `sum_two_numbers` and `subtract_two_numbers`. Now it is possible to execute callback from remote process:

```
from daffi import Global, FG, fetcher, __body_unknown__

@fetcher(FG)
def sum_two_numbers(a: int, b: int) -> int:
    __body_unknown__(a, b)


@fetcher(FG)
async def subtract_two_numbers(a: int, b: int) -> int:
    __body_unknown__(a, b)

g = Global(process_name='my awersome process', init_controller=True, host='localhost', port=8888)

result1 = sum_two_numbers(10, 15)
result2 = subtract_two_numbers(22, 13)
```

`FG` in this example is execution modifier. You can read about execution modifiers here

Warning

Do not use the same callback names in different processes unless you want to use `BROADCAST` or `STREAM` execution modifiers.

For singular execution, for instance if you have `sum_two_numbers` callback registered in process `A` and in process `B` then only one of them will be triggered. Daffi use random strategy to execute callback by name. You cannot control which one will be triggered.

Note

You still can use registered callback as regular python function. Next example works fine locally:

```
from daffi import callback

@callback
def my_callback(a, b, c) -> int:
    print(a, b, c)

my_callback(1, 2, 3)
```

instead of decorating functions you can create a decorated class:

```
from daffi import callback


@callback
class RemoteCallbackGroup:

    def method1(self):
        ...

    def method2(self):
        ...

    @staticmethod
```

```
    def method3():
        ...
```

In this case all public methods of decorated class become remote callbacks (`public` means without underscore in the beginning of name)

Class callback group initialization has some limitations. Read this article to know more.

3.4 fetcher decorator

fetcher decorator is decorator for create `pointers` to remote callbacks.

Lets say we declared callback with name `trigger_me` in process-1

process-1

```
from daffi import callback


@callback
def trigger_me(a: int, b: int):
    return a + b
# ...
```

Then in process-2 we can create pointer to it using `fetcher` decorator

process-2

```
from daffi import fetcher, FG, __body_unknown__

@fetcher(FG)
def trigger_me(a: int, b: int):
    __body_unknown__(a, b)
# ...
```

After Global object initialization we can trigger this pointer like it is local function

```
result = trigger_me(1, 2)
```

But instead of local execution remote callback `trigger_me` in `process-1` will be triggered and result will be delivered to `process-2`

Execution modifier `FG` is optional argument for `fetcher`. If it is not specified in `fetcher` arguments it should be provided during fetcher execution

process-2

```
from daffi import fetcher, FG, __body_unknown__

@fetcher
def trigger_me(a: int, b: int):
    __body_unknown__(a, b)
# ...

result = trigger_me(4, 5) & FG
```

By default function body for function, decorated with `fetcher` is not used. That is why we use `__body_unknown__` placeholder (which can be simply replaced with `pass` statement)

When function body is not used then all arguments provided during fetcher execution will be reflected to remote callback. It is important to keep callback/fetcher arguments consistency.

But one can specify `args_from_body` argument during fetcher initialization. In this case `fetcher`'s return statement will be used as arguments for callback

At this moment only tuple or single return value is valid syntax for fetcher with `args_from_body=True`.

process-2

```
from daffi import fetcher, FG

@fetcher(FG, args_from_body=True)
def trigger_me(a: int, b: int):
    return a * 5, b * 5
# ...

result = trigger_me(4, 5)
```

Note

In case remote callback expects only 1 argument you can return single value from fetcher.

3.5 Execution modifiers

Execution modifiers are a set of classes and methods that indicate how the remote request should be executed and how process executor expects to wait for computed result.

For instance lets consider following example:

```
from daffi import Global, FG, fetcher


@fetcher(FG)
def my_remote_callback():
    pass


g = Global()

args = (1, 2, 3)
kwargs = dict(foo='bar')

result = my_remote_callback(*args, **kwargs)
```

`& FG` in this example means process will wait until result is returned.

We can use `BG` modifier:

```
...
from daffi import Global, BG, fetcher


@fetcher(BG)
def my_remote_callback():
    pass


g = Global()

args = (1, 2, 3)
kwargs = dict(foo='bar')

future = my_remote_callback(*args, **kwargs)

# Do something
result = future.get()
```

`& BG` means wait result 'in background'. This modifier returns instance of `AsyncResult`. You can take result whenever you want without blocking main process execution.

Or we can say remote callback do not return result if we don't interested in it:

```
...
@fetcher(BG(return_result=False))
def my_remote_callback():
    pass

...
# Do smth locally
...
```

LIST OF AVAILABLE EXECUTION MODIFIERS:

Modifier class	Description	Optional arguments
`FG`	Stands for `foreground`. It means current process execution should be blocked until result of execution is returned. Another syntax: `g.call.<callback_name>(*args, **kwargs).fg(timeout=15)`	- `timeout`: The time to wait result. If exeeded `TimeoutError` will be thrown
`BG`	Stands for `background`. This modifier returns `AsyncResult` instance instead of result. Fits for long running tasks where caller execution cannot be blocked for a long time. Another syntax: `g.call.<callback_name>(*args, **kwargs).bg(timeout=15, eta=3)`	- `timeout`: The time to wait result. If exeeded `TimeoutError` will be thrown - `eta`: The time to sleep in the background before sending execution request - `return_result`: boolean flag that indicates if we want to receive result of remote callback execution.
`PERIOD`	Use for scheduling reccuring tasks or tasks which should be executed several times. Another syntax: `g.call.<callback_name>(*args, **kwargs).period(at_time=[datetime.utcnow().timestamp() + 3, datetime.utcnow().timestamp() + 10])`	- `at_time`: One timestamp or list of timestamps. Timestamps should be according to utc time and it should be timestamp in the future. This argument forces remote callback to be triggered one or more times when when timestamp == datetime.utcnow().timestamp - `period`: Duration in seconds to trigger remote callback on regular bases. One can provide either `at_time` argument or `period` argument in one request. Not both!
`BROADCAST`	Trigger all available callbacks on nodes by name. If `return_result` argument is set to True then aggregated result will be returned as dictionary where keys are node names and values are computed results. Another syntax: `g.call.<callback_name>(*args, **kwargs).broadcast(timeout=15, eta=3, return_result=True)`	- `timeout`: The time to wait result. If exeeded `TimeoutError` will be thrown - `eta`: The time to sleep in the background before sending execution request `return_result`: If provided aggregated result from all nodes where callback exist will be returned.
`STREAM`	Start streaming to one or more remote callbacks. Another syntax: `g.call.<callback_name>(< Iterable >).stream()`	

3.6 Basic example

To give you a flavour of how it works let's create simple application.

This application will consist of two python files `publisher.py` and `consumer.py`.

- `publisher.py` will register a simple function `sum_of_two_numbers` as a remote callback. This function should take 2 numbers and return their sum.
- `consumer.py` will call the function `sum_of_two_numbers` remotely passing it 2 numbers as arguments, wait for the result and print it.

publisher.py

In `publisher.py` we need to create function `sum_of_two_numbers` and make it visible for `consumer.py`. For this reason callback decorator is used.

```
import logging
from daffi import Global, callback

logging.basicConfig(level=logging.INFO)


@callback # (1)
def sum_of_two_numbers(a: int, b: int) -> int:
    return a + b


if __name__ == '__main__':

    g = Global(host="localhost", port=8888, init_controller=True) # (2)
    g.join() # (3)
```

(This script is complete, it should run "as is")

- `(1)` we use `callback` decorator which registers the function as a remote callback. It makes `sum_of_two_numbers` visible to all other processes where daffi is running. More details about using `callback` decorator you can find here.
- `(2)` we initialize `Global` object. To know more about `Global` follow this link. It is worth to mention that we use `init_controller=True` here. Among several daffi processes we need to have one where `Controller` is initialized. `Controller` in daffi teminology is server. `Controller` can be stand-alone process or be combined with `Node` (as in this example). More details about nodes and controllers here
- `(3)` we join Global instance to main thread. By default Global runs daffi components in separate thread.

---

consumer.py

In `consumer.py` we need to describe how to execute `sum_of_two_numbers` registered in `publisher.py`. For this reason fetcher is used. You can initialize and execute fetcher in different ways. Les't go trough some of them:

```
import logging
from daffi import Global, FG, fetcher, __body_unknown__

logging.basicConfig(level=logging.INFO)


@fetcher(exec_modifier=FG) # (1)
def sum_of_two_numbers(a: int, b: int) -> int:
    __body_unknown__(a, b)


if __name__ == '__main__':

    g = Global(host="localhost", port=8888) # (2)

    result = sum_of_two_numbers(5, 15) # (3)
    print(f"Result = {result}")

    g.join()
```

(This script is complete, it should run "as is")

- (1) we register fetcher as pointer to remote sum_of_two_numbers callback.
  Argument exec_modifier=FG means each time we trigger fetcher we want to wait for result. More details about available execution modifiers here
  In this particular example decorated function name and signature must be the same as remote callback has.
  In function body we are using __body_unknown__ mock. fetcher doesn't use function body by default. Only function name and function signature (arguments) are used to trigger remote. Feel free to use pass statement instead of __body_unknown__.
- (2) we initialize Global. Port and host must correspond to port and host of Controller that we initialized in publisher.py.
- (3) we trigger sum_of_two_numbers fetcher. Arguments passed to this fetcher will be transferred to publisher.py process where callback sum_of_two_numbers will be triggered.

To check how it works start publisher.py and consumer.py in two separate terminals

```
python3 publisher.py
python3 consumer.py
```

There is also another argument args_from_body you can pass to fetcher. In this case result returned from fetcher's body is using as arguments to pass to remote callback.

Let's modify consumer.py:

```
import logging
from typing import Tuple
from daffi import Global, FG, fetcher

logging.basicConfig(level=logging.INFO)


@fetcher(exec_modifier=FG, args_from_body=True) # (1)
def sum_of_two_numbers(multiplier: int) -> Tuple[int, int]:
    arg1 = 5 * multiplier
    arg2 = 10 * multiplier
    return arg1, arg2


if __name__ == '__main__':

    g = Global(host="localhost", port=8888)

    result = sum_of_two_numbers(multiplier=2)
    print(f"Result = {result}")

    g.join()
```

(This script is complete, it should run "as is")

- (1) we register fetcher with args_from_body=True argument.
  In this example only decorated function name should be the same as name of remote callback. If args_from_body=True argument is provided then we can use any arguments want.
  But return statement of decorated function must return tuple of 2 values (a: int, b: int) that corresponds to arguments specified in callback sum_of_two_numbers signature.

> **Note**
>
> In case remote callback expects only 1 argument you can return single value from fetcher.

To check how it works start publisher.py and consumer.py in two separate terminals

```
python3 publisher.py
python3 consumer.py
```

Note

You can modify `publisher.py` and `consumer.py` scripts to try UNIX socket istead of connection via host/port. For this just delete host and port arguments from `Global` object initialization in both scripts.

When host and port are not provided UNIX socket is default connection.

Another fetcher syntax examples

fetcher is quite flexible.

You can specify `exec_modifier` argument in decorator or skip it completely.

In this case you should provide execution modifier during `fetcher` execution.

This way is appropriate when you need to trigger remote callback with different execution modifiers depends on situation.

```
import logging
from daffi import Global, fetcher, __body_unknown__, FG, BROADCAST, STREAM

logging.basicConfig(level=logging.INFO)

stream_values = [("a", "b"), ("c", "d"), ("f", "g"), ("y", "z")]


@fetcher
def my_awersome_func(a: str, b: str) -> str:
    __body_unknown__(a, b)


if __name__ == '__main__':

    g = Global(host="localhost", port=8888)

    # Trigger one `my_awersome_func` callback on remote and wait for result
    result = my_awersome_func(a="foo", b="bar") & FG
    print(f"Result = {result}")

    # Trigger all `my_awersome_func` callbacks registered on remote nodes (broadcast)
    my_awersome_func(a="foo", b="bar") & BROADCAST

    # Stream to all `my_awersome_func` callbacks registered on remote nodes
    my_awersome_func(stream_values) & STREAM
    g.join()
```

Note

execution modifiers take various arguments and can be passed to fetcher as class or class instance. For example `BG` execution modifier has 2 arguments `timeout` and `eta`. `timeout` is used when you want to limit execution time on remote. If this value exceeded then `TimeoutError` will be thrown. `eta` describes delay in seconds before remote callback execution.
So you can consider two options:

```
from daffi import Global, fetcher, BG

@fetcher(BG)
def my_awersome_func(a: str, b: str) -> str:
    __body_unknown__(a, b)
...
# BG execution modifier returns instance of `AsyncResult`. To obtain result we need to use .get() method of `AsyncResult`
future = my_awersome_func(1, 2)
result = future.get()
```

(fetcher without timeout and without eta.)

```
from daffi import Global, fetcher, BG

fetcher(BG(timeout=10, eta=2))
def my_awersome_func(a: str, b: str) -> str:
    __body_unknown__(a, b)
...
# BG execution modifier returns instance of `AsyncResult`. To obtain result we need to use .get() method of `AsyncResult`
future = my_awersome_func(1, 2)
result = future.get()
```

(fetcher waits for result no more then 10 seconds and trigger callback execution after 2 second of delay(eta))

You can also trigger execute remote callback through Global object without fetcher registration. General syntax can be described as the following:

```
g.call.<remote callback name>(*args, **kwargs) & <execution modifier>
```

where `<remote callback name>` is the name of function registered in different process, `*args` and `**kwargs` is any arguments you want to pass to this function and `<execution modifier>` is specific class that describes how to execute remote (single call, stream, broadcast, scheduled trigger etc) and how to wait for result (foregraund, background etc)

Example:

```
import logging
from daffi import Global, FG

logging.basicConfig(level=logging.INFO)

if __name__ == '__main__':

    g = Global(host="localhost", port=8888)

    result = g.call.my_awersome_func(a="foo", b="bar") & FG
    print(f"Result = {result}")

    g.join()
```

This syntax has 2 problems:

- If your application is large, then you need to have the `g` object available in all places where remote callbacks are called.
- This syntax is not IDE friendly. IDE can't autocomplete or suggest you available callbacks on remote processes.

But it still can be considered for tiny microservices architecture.

3.7 Example with bidirectional communication

In previous example we considered only `consumer` to `publisher` communication. In other words only `consumer` triggered remote callbacks.

Lets modify `publisher.py` and `consumer.py` so that each of these processes registers a callback and triggers a callback from other process.

For example lets keep `sum_of_two_numbers` callback in `publisher.py` but create new function `consumer_time` in `consumer.py`. `consumer_time` should just return current UTC timestamp of consumer process.

publisher.py

```
# publisher.py
import asyncio
import logging
from daffi import Global, callback, fetcher, __body_unknown__, FG

logging.basicConfig(level=logging.INFO)


@callback  # (1)
def sum_of_two_numbers(a: int, b: int) -> int:
    return a + b


@fetcher(FG)
async def consumer_time() -> float:
    __body_unknown__()


async def runner(): # (2)
    for _ in range(10):
        current_consumer_time = consumer_time()
        print(f"Current consumer time: {current_consumer_time}")

        await asyncio.sleep(3)


if __name__ == '__main__':
    g = Global(host="localhost", port=8888, init_controller=True, process_name="publisher")  # (3)

    g.wait_process("consumer") # (4)

    asyncio.run(runner()) # (5)

    g.stop()
```

(This script is complete, it should run "as is")

- (1) Here we registered `sum_of_two_numbers` function as remote callback. Syntax is the same as in previous examples.
- (2) `runner` is async function that will be running in asyncio event loop. This function triggers `consumer_time` remote callback that is registered in `consumer.py` process. Execution happen to be 10 times in the cycle.
- (3) Here we initialize `Global` object. In additional to provided host and port we also assign specific process name. This way we can wait process each other by name.
- (4) Here we wait for `consumer` process to be started.
- (5) Start main `runner` function here

consumer.py

```
# consumer.py
import asyncio
import logging
from datetime import datetime
from daffi import Global, FG, fetcher, __body_unknown__, callback

logging.basicConfig(level=logging.INFO)


@callback # (1)
async def consumer_time() -> float:
    return datetime.utcnow().timestamp()


@fetcher(FG)  # (2)
async def sum_of_two_numbers(a: int, b: int) -> int:
```

```
    __body_unknown__(a, b)


async def runner():  # (3)
    for _ in range(10):
        result = sum_of_two_numbers(5, 15)
        print(f"Result = {result}")

        await asyncio.sleep(3)


if __name__ == "__main__":

    g = Global(host="localhost", port=8888, process_name="consumer")  # (4)

    g.wait_process("publisher")  # (5)

    asyncio.run(runner())  # (6)

    g.stop()
```

(This script is complete, it should run "as is")

- (1) Here we registered `consumer_time` function as remote callback.
- (2) Here we created fetcher for `sum_of_two_numbers` function that is registered as callback in `publisher.py` process.
- (3) `runner` is async function that will be running in asyncio event loop. This function triggers `sum_of_two_numbers` remote callback that is registered in `publisher.py` process. Execution happen to be 10 times in the cycle.
- (4) Here we initialize `Global` object. In additional to provided host and port we also assign specific process name. This way we can wait process each other by name.
- (5) Here we wait for `publisher` process to be started.
- (6) Start main `runner` function here

> **Note**
>
> Daffi works fine with both synchronous and asynchronous functions but some methods blocks event loop so daffi has async method options for such situations.
>
> Read more here if your application leverage on event loop.

Sometimes you need to register specific callback to make it available to call from remote processes but you also want to call callback with the same name registered on remote (aka celery style).

It is common practice in dockerized applications to have one code base but different entrypoint commands. For such situations you can register your function as `callback` and `fetcher` at the same time

Example:

```
import logging
from daffi import fetcher, callback

logging.basicConfig(level=logging.INFO)

@fetcher
@callback
async def my_callback() -> str:
    # Do something
    return "abcdefg"

...
```

There is another decorator `callback_and_fetcher` which you can use if you want your function to be combination of `fetcher` and `callback`

Lets modify `publisher.py` and `consumer.py` from previous example so that both of them have one function `get_process_name` which returns name of process.

publisher.py

```python
# publisher.py
import asyncio
import logging
from daffi import Global, callback_and_fetcher, FG

logging.basicConfig(level=logging.INFO)

PROCESS_NAME = "publisher"


@callback_and_fetcher  # (1)
async def get_process_name() -> str:
    return PROCESS_NAME


async def runner():  # (2)
    for _ in range(10):
        remote_process_name = get_process_name() & FG
        print(f"{PROCESS_NAME} called {remote_process_name!r}")

        await asyncio.sleep(3)


if __name__ == "__main__":
    g = Global(host="localhost", port=8888, init_controller=True, process_name=PROCESS_NAME)

    g.wait_process("consumer")

    asyncio.run(runner())

    g.stop()
```

- (1) We registered `get_process_name` function as `fetcher` and `callback`. It means this function is visible to other processes by name due to it is `callback` but if we trigger it locally it works as `fetcher` (trigger remote callback with name `get_process_name` on remote)
- (2) We declared runner function which triggers `get_process_name` 10 times in cycle.

consumer.py

```python
# consumer.py
import asyncio
import logging
from daffi import Global, callback_and_fetcher, FG

logging.basicConfig(level=logging.INFO)

PROCESS_NAME = "consumer"


@callback_and_fetcher
async def get_process_name() -> str:
    return PROCESS_NAME


async def runner():
    for _ in range(10):
        remote_process_name = get_process_name() & FG
        print(f"{PROCESS_NAME} called {remote_process_name!r}")

        await asyncio.sleep(3)


if __name__ == "__main__":

    g = Global(host="localhost", port=8888, process_name="consumer")

    g.wait_process("publisher")

    asyncio.run(runner())

    g.stop()
```

`consumer.py` does pretty much the same as `publisher.py`. The only difference then `publisher.py` initializes controller and node whereas `consumer.py` only node.

This way you can have only one common script instead of `publisher.py` and `consumer.py`. The only thing you should do is to pass env variables `PROCESS_NAME` and `INIT_CONTROLLER` (True or False) during initialization.

You can check another examples at: https://github.com/600apples/dafi/tree/main/examples

4. Node and Controller

4.1 Controller

In daffi terminology server is called `Controller`.

The `Controller` behaves as a broker and cannot call remote callbacks on its own. `Controller` can be run in any of the processes where remote callbacks are registered, but it can also work as stand-alone application. Both variants are fine depends on your requirements. Usually application infrastructure includes only one `Controller` but it is possible to have several if you want to create several isolated systems.

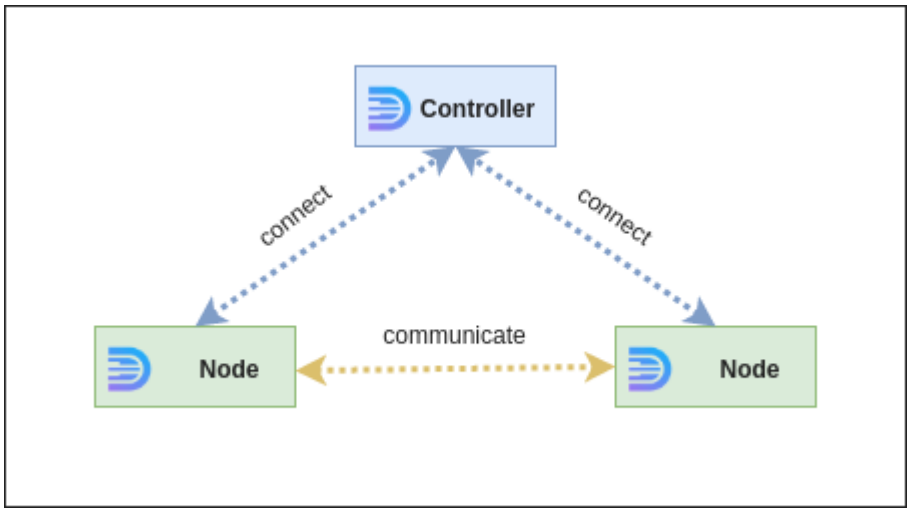
4.2 Node

`Node` is client that is running along with `Controller` or as stand-alone process.

The application where the `Node` is running gets the opportunity to register callbacks and call the callbacks of other nodes. All serialization/deserialization process, remote callbacks executions etc is the `Node`'s responsibility.

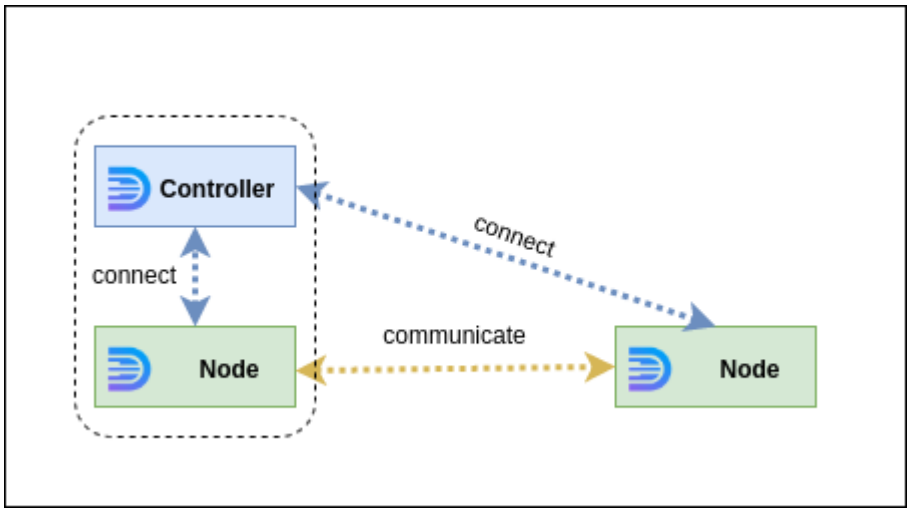
4.2.1 Typical architecture

A typical application architecture consists of one controller and two or more nodes. The controller can work as a standard application or share the process with a node:



Stand alone `Controller`.

This architecture is suitable where there are more than 2 nodes and complex inter-node communication is expected.



`Controller` shares process with one of nodes.

This architecture is suitable for simple node-to-node requests or streams. But also can be considered as solution when one of the processes is the leader with the ability to give commands to all other nodes

Warning

Again. `Controller` is just broker for inter-node communication.

If you want to initialize controller with the ability to make remote requests, you need to consider latter solution (`Controller shares process with one of nodes`)

5. Asynchronous applications

Most of daffi methods works fine with both sync and async applications but some of them blocks event loop.

Blocking methods have async conterparts which should be used for asynchronous processes.

For example Global object has method `join` which is blocking method and cannot be used for applications which leverage on event loop.

But `Global` also has `join_async` method which fits async non blocking model.

Also there are two methods `get` and `get_async` to take result depending on application type.

As example we can use `BG` execution modifier (run in background) which returns instance of `AsyncResult` instead of result itself:

```
from daffi import fetcher, BG

@fetcher(BG)
def my_callback():
    pass

future = my_callback()

# get result in blocking manner
result = future.get()

# get result in async applications
result = await future.get_async()
```

Despite on remote callback type we can declare synchronous or asynchronous fetcher depends on our application.

For instance if in `process-1` we have callback with name `trigger_me`:

process-1

```
from daffi import callback

@callback
def trigger_me():
    print("Triggered")
```

In `process-2` we can declare sync or async fetcher to trigger callback:

process-2

```
from daffi import fetcher, FG

@fetcher(FG)
def trigger_me():
    pass

def main():
    trigger_me()

# ....
```

The more appropriate syntax for asynchronous applications will be:

process-2

```
from daffi import fetcher, FG

@fetcher(FG)
async def trigger_me():
    pass

async def main():
    await trigger_me()

# ....
```

6. Advanced

6.1 G object injection

One can create and register callback that takes special argument `g`

```
from daffi import callback, Global

@callback
def my_callback(g: Global, **kwargs):
    ...
```

Warning

`g` is reserved argument name and cannot be used to pass arbitrary data

In fact `g` argument works like fixture in pytest. Once `g` is specified as argument on remote callback it will be automatically populated with Global instance on remote process.

You should never pass Global object directly as argument. Instead just specify that remote callback requires this argument.

Such behavior is helpful when you consider to build pipeline architecture.

6.2 Callback classes

Instead of registering a single remote callback by decorating a function, you can decorate a class. In this case, all public methods of the class become callbacks.

Lets consider we have Node `node-1` with CallbackGroup registered as callback

```
from daffi import callback


@callback
class CallbackGroup:
    """All public methods (without '_') become callbacks."""

    def __init__(self):
        self.internal_value = "my secret value"

    def method1(self):
        return f"triggered method1. Internal value: {self.internal_value}"

    def method2(self, *args, **kwargs):
        return f"triggered method2 with arguments: {args}, {kwargs}"

    @classmethod
    def class_method(cls, *args, **kwargs):
        return f"triggered classmethod with arguments: {args}, {kwargs}"

    @staticmethod
    def static_method(*args, **kwargs):
        return f"triggered staticmethod with arguments: {args}, {kwargs}"

    def _private_method(self):
        pass
```

This method of registration has certain limitations.

- Only methods that are class methods and statics methods are visible callbacks by default as to trigger static or class method there is no neeed to instantiate object of class.

Now, If we try to execute `method1` we got `GlobalContextError` error

```
from daffi import Global, FG, fetcher

proc_name = "node-2"


@fetcher(FG)
def method1():
    pass

g = Global(init_controller=True)

g.wait_process('node-1')

result = method1()

# ---- traceback ----
# GlobalContextError(
# daffi.exceptions.RemoteCallError:
# Exception while processing function 'method1' on remote executor.
# Instance of 'CallbackGroup' is not initialized yet.
# Create instance or mark method 'method1' as classmethod or staticmethod
```

- second limitation is you can have only one instance of `CallbackGroup` created

Next code will thrown `InitializationError`

```
cb1 = CallbackGroup()
cb2 = CallbackGroup()

# ---- traceback ----
#  InitializationError(f"Only one callback instance of {class_name!r} should be created.")
#  daffi.exceptions.InitializationError: Only one callback instance of 'CallbackGroup' should be created.
```

- and last limitation is you cannot decorate particular methods of class, only class itself!

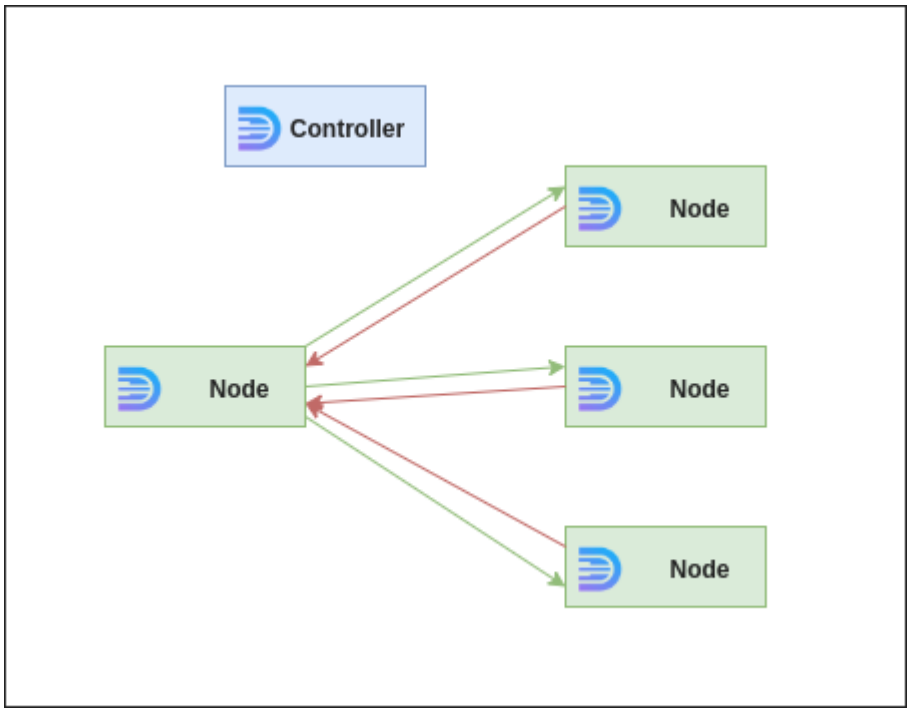
Note

You still can use `CallbackGroup` instance as regular class instance after initialization. For example next code is valid:

```
cb1 = CallbackGroup()

cb1.method1()
cb1.method2("foo", "bar")
cb1.static_method()
```

6.3 Broadcasting



Broadcasting illustration.

It is possible to register callback with the same name on many nodes.

Let's consider we have the following callback registered on nodes with names `node-1`, `node-2` and `node-3`

```
from daffi import Global, callback

proc_name = "node-<1, 2, 3>" # node-1, node-2 or node-3 depends on node

@callback
def my_callback(some_string: str) -> str:
    return some_string + "-" + proc_name


g = Global() # Connection via UNIX socket
g.join()
```

After that we can trigger broadcast remote call from node-4:

```
from daffi import Global, BROADCAST, fetcher, __body_unknown__

proc_name = "node-4"

@fetcher(BROADCAST(return_result=True))
def my_callback(some_string: str) -> str:
    __body_unknown__(some_string)


g = Global(init_controller=True)

# Wait all processes to start
for proc in ("node-1", "node-2", "node-3"):
    g.wait_process(proc)

result = my_callback(some_string="abc")

print(result)
```

You should see the following output:

```
{
"node-1": "abc-node-1",
"node-2": "abc-node-2",
```

```
"node-3": "abc-node-3"
}
```

Where keys of dictionary are referred to node names where result was calculated and values represents calculated results respectively.

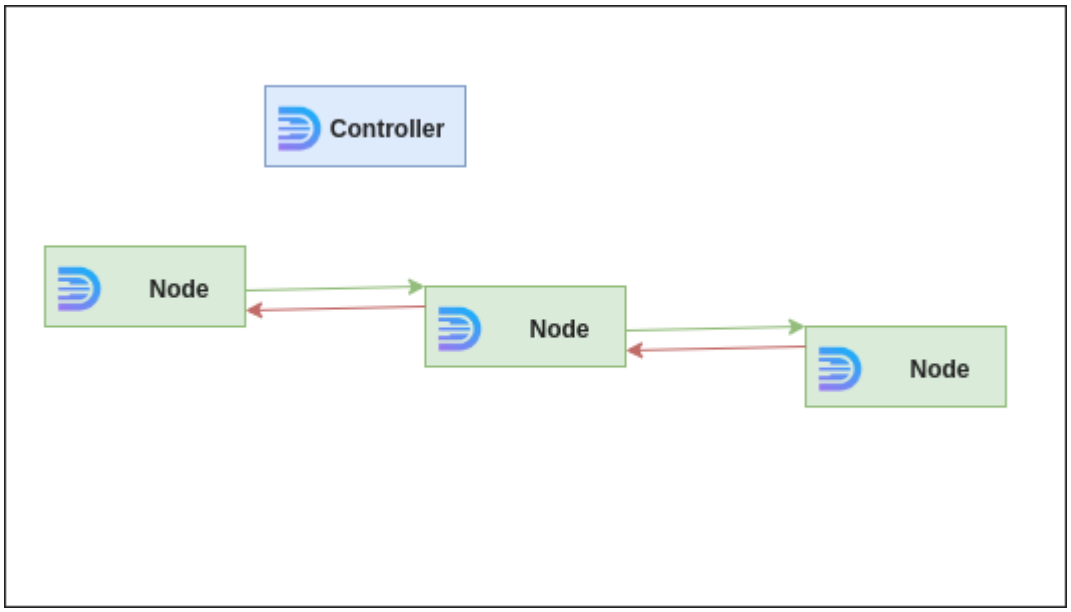
One can also use g style callback execution instead of `fetcher`. For instance following statements have the same effect:

```
result = g.call.my_callback(some_string="abc").broadcast(return_result=True)
```

or

```
result = g.call.my_callback(some_string="abc") & BROADCAST(return_result=True)
```

6.4 Pipeline and g object injection



Pipeline illustration.

It is possible to trigger chain of callbacks where one callback takes result from another and returns this result to caller.

For this feature there is special ability of daffi we didn't mentioned before:

One can create and register callback that takes special argument `g` (You can read more about g object injection)

process `node-1` :

```
from daffi import callback, Global, FG

@callback
def pipeline_cb1(g: Global):
    res = g.call.pipeline_cb2() & FG
    return res
```

Following this example lets consider we have `pipeline_cb2` registered on `node-2` and `pipeline_cb3` registered on `node-3`

process `node-2` :

```
from daffi import callback, Global, FG

@callback
def pipeline_cb2(g: Global):
    res = g.call.pipeline_cb3() & FG
    return res
```

process `node-3` :

```
from daffi import callback

@callback
def pipeline_cb3():
    return "my secret sting"
```

So if we execute `pipeline_cb1` from separate process:

```
from daffi import Global, FG, fetcher

proc_name = "node-4"

@fetcher(FG)
def pipeline_cb1():
    pass
```

```
g = Global(init_controller=True)

# Wait all processes to start
for proc in ("node-1", "node-2", "node-3"):
    g.wait_process(proc)

result = pipeline_cb1()

print(result)
```

we got "my secret sting" as the result of execution but not directly from `pipeline_cb1`. Full chain is `pipeline_cb1` -> `pipeline_cb2` -> `pipeline_cb3`

6.5 Scheduling tasks

It is possible to start reccuring callback execution or set bunch of delayed callback executions at once.

For this reason special class modifier `PERIOD` is used.

`PERIOD` take 2 optional arguments `at_time` and `interval` but only one of them is allowed to be provided.

AT_TIME

- `at_time` - take one timestamp or list of timestamps. They all should be timestamps related to UTC time (and timestamps in the future).

For instance lets consider you want to execute remote callback `my_callback` 3 times, After 2 seconds after 10 seconds and after a minute:

```
from datetime import datetime
from daffi import Global, PERIOD, fetcher

g = Global(host="localhost", port=8888)


# Here we registered fetcher without execution modifier. If execution modifier not provided then it should be specified
# during fetcher execution using syntax <fetcher>(*args, **kwargs) & <execution modifier>
@fetcher
def some_func():
    pass
# Wait my_callback function to be available on remote
g.wait_function("some_func")

now = datetime.utcnow().timestamp()

# 2 sec, 10 sec and 60 sec later
at_time = [now + 2, now + 10, now + 60]

task = some_func() & PERIOD(at_time=at_time)
```

Execution with `PERIOD` modifier returns `ScheduledTask` instance as result of execution.

You can cancel all tasks triggered from this execution:

```
...
at_time = [now + 2, now + 10, now + 60]
task = some_func() & PERIOD(at_time=at_time)

... # do smth
time.sleep(3)
task.cancel()
```

As we slept 3 seconds before canceling only two executions that remains in `at_time` bunch will be canceled as first of them was already triggered earlier.

INTERVAL

Using `interval` has the same execution signature and also returns instance of `ScheduledTask` as result so you can cancel reccuring task any time you want

```
import time
from datetime import datetime
from daffi import Global, PERIOD, fetcher


@fetcher(PERIOD(interval=5))
def some_func():
    pass

g = Global(host="localhost", port=8888)

# Wait my_callback function to be available on remote
g.wait_function("some_func")

now = datetime.utcnow().timestamp()

# Execute remote callback `some_func` each 5 seconds
task = some_func()
time.sleep(60)
task.cancel()
```

On example above we started recurring execution each 5 second and canceled it after sleep

Note

interval also takes special string formatted expressions as values for instance next statement is also valid:

PERIOD(interval="5s")

Another examples:

PERIOD(interval="1m24s") # == 84 seconds

PERIOD(interval="1.2 minutes") # == 72 seconds

6.6 Streams

Stream is Daffi's special ability to pass iterables to remote callbacks. It is very optimized for huge bandwidth.

A stream can be considered as a union of `BROADCAST` and `BG(no_return=True)` execution modifiers.

For instance we have remote callback which takes one `int` as argument and store this argument to global list and we need to pass range of integers from 1 to 100 from another `Node`.

(Of course you can pass full range to callback at once but lets imagine you have unknown range or range that can be infinity generator)

process 1

```
... # Global object initialization is omitted.

from daffi import callback

sequence_store = []

@callback
def process_sequence(a: int) -> None:
    global sequence_store
    sequence_store.append(a)
```

process 2

```
from daffi import Global, fetcher, __body_unknown__, BG


@fetcher(BG(no_return=True))
def process_sequence(a: int) -> None:
    __body_unknown__(a)


g = Global(host='localhost', port=8888)

for i in range(1, 101):
    process_sequence(i)
```

Or let's even say we have `process_sequence` callback registered on many nodes and we need to pass range to all of them:

process 2

```
from daffi import Global, BROADCAST, fetcher, __body_unknown__


@fetcher(BROADCAST)
def process_sequence(a: int) -> None:
    __body_unknown__(a)


g = Global(host='localhost', port=8888)

for i in range(1, 101):
    process_sequence(i)
```

The 2 examples above work as they should, but there is a more optimized `STREAM` execution modifier:

process 2

```
from daffi import Global, STREAM, fetcher, __body_unknown__


@fetcher(STREAM)
def process_sequence(a: int) -> None:
    __body_unknown__(a)


g = Global(host='localhost', port=8888)

integers_range = range(1, 101)

process_sequence(integers_range)
```

Callbacks registered for streams have only one limitation. The should take exactly 1 argument This argument becomes stream item on each iteration of stream process.

Note

You still can pass many arguments using stream of tuples or stream of dictionaries

process 1

```
... # Global object initialization is omitted.

from daffi import callback


@callback
def process_stream(args: tuple) -> None:
    arg1, arg2, arg3 = args
    ...
```

process 2

```
from daffi import Global, STREAM, fetcher


@fetcher(STREAM)
def process_stream(sequence_item)
    pass

g = Global(host='localhost', port=8888)

arguments = [(1, 2, "a"), (3, 4, "b"), (5, 6, "c")]

process_stream(arguments)
```

You might be wondering why not just use `BROADCAST` and `BG` execution modifiers in the cycle?

Answering briefly `STREAM` is much more optimized for passing big ranges to remote callback whereas `BROADCAST`, `FG`, `BG` modifiers are more optimized for single callback execution

EXAMPLE WITH OPENCV CAMERA STREAM

Warning

This example was tested on Ubuntu22. It doens't work as expected on Mac since limitation of using opencv's method `imshow` in threads for this OS.

Note

You need to install opencv-python for this example

Lest consider more realistic usage of stream. Imagine you have one node that read video stream from camera and you need to show this stream in two other nodes.

Process 1 is stream generator

process 1

```
import asyncio
import logging
import cv2
from daffi import Global, STREAM, fetcher, __body_unknown__

logging.basicConfig(level=logging.INFO)

cap = cv2.VideoCapture(0)


@fetcher(STREAM)
async def show_stream(frame: int) -> None:
    """Display the resulting image frame"""
```

```
    __body_unknown__(frame)


def frame_iterator():
    """Read frame from camera and send it to all receivers"""

    # Percent of original size
    # Images with high resolution might cause stream throttling
    # It depends on your camera. Adjust this value if needed.
    scale_percent = 20
    ret, frame = cap.read()
    while ret:
        ret, frame = cap.read()
        width = int(frame.shape[1] * scale_percent / 100)
        height = int(frame.shape[0] * scale_percent / 100)
        dim = (width, height)
        resized = cv2.resize(frame, dim, interpolation=cv2.INTER_AREA)
        yield resized


async def main():
    with Global(init_controller=True, host="localhost", port=8888) as g:

        for proc in ('process2', 'process3'):
            g.wait_process(proc)

        show_stream(frame_iterator())

    cap.release()
    cv2.destroyAllWindows()


if __name__ == "__main__":
    asyncio.run(main())
```

(This script is complete, it should run "as is")

Process 2 and Process 3 are stream consumers

process 2

```
import cv2
import logging
from daffi import Global, callback

logging.basicConfig(level=logging.INFO)


@callback
async def show_stream(frame: int) -> None:
    """Display the resulting image frame"""
    cv2.imshow("daffi stream consumer 1", frame)
    cv2.waitKey(1)


def main():
    g = Global(process_name='process2', host="localhost", port=8888)
    g.join()


if __name__ == "__main__":
    main()
```

(This script is complete, it should run "as is")

process 3

```
import cv2
import logging
from daffi import Global, callback

logging.basicConfig(level=logging.INFO)


@callback
async def show_stream(frame: int) -> None:
    """Display the resulting image frame"""
    cv2.imshow("daffi stream consumer 2", frame)
    cv2.waitKey(1)


def main():
    g = Global(process_name='process3', host="localhost", port=8888)
    g.join()


if __name__ == "__main__":
    main()
```

(This script is complete, it should run "as is")

Then in 3 separate terminals:

```
python process1.py
python process2.py
python process3.py
```

You should see 2 windows with camera stream on your machine

7. Code reference

7.1 Global

Main daffi entrypoint for all remote operations eg. call remote callbacks, wait for remote processes, obtain additional information from remote etc.

Parameters:

Name	Type	Description	Default
`process_name`	`Optional[str]`	Global process name. If specified it is used as reference key to Node process. By default randomly generated hash is used as reference.	`field(default_factory=string_uuid)`
`init_controller`	`Optional[bool]`	Flag that indicates whether `Controller` should be instantiated in current process	`False`
`init_node`	`Optional[bool]`	Flag that indicates whether `Node` should be instantiated in current process	`True`
`host`	`Optional[str]`	host to connect `Controller` / `Node` via tcp. If not provided then Global consider UNIX socket connection to be used.	`None`
`port`	`Optional[int]`	Optional port to connect `Controller` / `Node` via tcp. If not provided random port will be chosen.	`None`
`unix_sock_path`	`Optional[os.PathLike]`	Folder where UNIX socket will be created. If not provided default path is < tmp directory >/dafi/ where `<tmp directory >` is default temporary directory on system.	`None`

7.1.1 call: LazyRemoteCall property

Returns instance of `LazyRemoteCall` that is used to trigger remote callback.

7.1.2 is_controller: bool property

Return True if controller is running in current process

7.1.3 registered_callbacks: Dict[str, List[str]] property

Return list of all registered callbacks along with process names where these callbacks are registered.

7.1.4 cancel_scheduled_task_by_uuid(remote_process: str, uuid: int) -> NoReturn

Cancel scheduled task by its uuid on remote process. Find out task uuid you can using `get_scheduled_tasks` method.

7.1.5 get_scheduled_tasks(remote_process: str)

Get all scheduled tasks that are running at this moment on remote process

Parameters:

Name	Type	Description	Default
remote_process	str	Name of Node	*required*

7.1.6 join() -> NoReturn

Join global to main thread. Don't use this method if you're running asynchronous application as it blocks event loop.

7.1.7 join_async() async

Async version of .join() method. Use this method if your application is asynchronous.

7.1.8 kill_all()

Kill all connected nodes. This method works only for those processes where controller is running

7.1.9 stop(kill_all_connected_nodes: Optional[bool] = False)

Stop all components (Node/Controller) that is running are current process

Parameters:

Name	Type	Description	Default
kill_all_connected_nodes	Optional[bool]	Optional flag that indicated whether kill signal should be sent to all connected nodes. This argument works only for those processes where controller is running.	False

7.1.10 transfer_and_call(remote_process: str, func: Callable[..., Any], *args: Tuple[Any], **kwargs: Dict[Any, Any]) -> Union[Coroutine, Any]

Send function along with arguments to execute on remote process. This method has some limitations. For example you should import modules inside function as modules might be unavailable on remote.

Parameters:

Name	Type	Description	Default
remote_process	str	Name of node where function should be executed	*required*
args	Tuple[Any]	Any positional arguments to execute function with.	()
kwargs	Dict[Any, Any]	Any keyword arguments to execute function with.	{}

Example

```
import os

from daffi import Global

async def get_remote_pid(): import os return os.getpid()

g = Global() g.transfer_and_call("node_name", get_remote_pid)
```

7.1.11 wait_function(func_name: str) -> NoReturn

Wait particular remote callback by name to be available. This method is useful when callback with the same name is registered on different nodes and you need at leas one node to be available.

Parameters:

Name	Type	Description	Default
func_name	str	Name of remote callback to wait.	*required*

7.1.12 wait_function_async(func_name: str) -> NoReturn `async`

Wait particular remote callback by name to be available (async version). This method is useful when callback with the same name is registered on different nodes and you need at leas one node to be available.

Parameters:

Name	Type	Description	Default
func_name	str	Name of remote callback to wait.	*required*

7.1.13 wait_process(process_name: str) -> NoReturn

Wait particular Node to be alive.

Parameters:

Name	Type	Description	Default
process_name	str	Name of `Node`	*required*

7.1.14 wait_process_async(process_name: str) -> NoReturn `async`

Wait particular Node to be alive (async version).

Parameters:

Name	Type	Description	Default
process_name	str	Name of `Node`	*required*

7.2 callback

Bases: `Decorator`

callback is uniform decorator for registering remote callbacks from functions or from classes

Example

```
from daffi import callback

@callback def my_func(args, *kwargs): ...
```

lback works with classes but some limitations are present

1. Only `static` methods and `class` methods can be triggered without class initialization. For all other methods to be instance of class should be instantiated.
2. Only one instance of class can be instantiated.
3. Only publicly available methods become callback (means methods which name doensn't start with underscore)

7.3 fetcher

Bases: `Decorator`

fetcher is decorator converts a decorated method or class into a remote object call. Decorated function cannot be used for local execution. Essentially, the 'fetcher' decorator is an alternative syntax to `g.call.(*args*, *kwargs) * < exec modifier>. which is more friendly to IDE.

Lets consider you have function with name and signature `my_awersome_function(a: int, b: int, c: str): ...` registered on one of nodes. Yo need to create fetcher with the same name and the same signature to call function on remote:

Example

from daffi import fetcher, **body_unknown**, FG

@fetcher def my_awersome_function(a: int, b: int, c: str): # or use pass. Internal logic will be skipped in any case. only name and signature is important **body_unknown**(a, b, c)

7.3.1 Then we can call `my_awersome_function` on remote

result = my_awersome_function(1, 2, "abc") & FG

Or if you want to bind execution modifier to fetcher:

from daffi import fetcher, **body_unknown**, FG

@fetcher(FG) def my_awersome_function(a: int, b: int, c: str): # or use pass. Internal logic will be skipped in any case. only name and signature is important **body_unknown**(a, b, c)

7.3.2 Then we can call `my_awersome_function` on remote

7.3.3 !!! Execution modifier is binded to fetcher. No need to use `& FG` after execution.

result = my_awersome_function(1, 2, "abc")